

从失败轨迹到可靠的大语言模型智能体：诊断与修复框架缺陷

Mengzhuo Chen^{1,2,3}, Junjie Wang^{1,2}, Zhe Liu^{1,2}, Yawen Wang^{1,2}, Haiming Zheng⁴, Qing Wang^{1,2}

¹State Key Laboratory of Complex System Modeling and Simulation Technology, Beijing, China

²Institute of Software, Chinese Academy of Sciences, Beijing, China

³University of Chinese Academy of Sciences, Beijing, China

⁴School of Computer Science and Technology, Tianjin University, Tianjin, China

摘要——大语言模型智能体日益依赖智能体框架：即基础模型周围的运行时基础设施，它定义了执行环境、工具接口、上下文、生命周期编排、可观测性、验证与治理。现有的自改进智能体与自动框架演化方法主要通过运行时监督、提示优化、工作流搜索或基于最终结果的框架修改来改进智能体。然而，它们往往无法诊断失败轨迹中责任证据所在的位置，以及哪个框架实现机制导致了不可靠行为，从而导致宽泛、间接或范围不当的更改。本文提出 HARNESSEXFIX，一个基于轨迹定位和诊断驱动的智能体框架修复框架。HARNESSEXFIX 将原始执行轨迹和框架工件编译为框架感知的轨迹中间表示（HTIR），该表示规范化碎片化的轨迹证据，捕获步骤级数据流和控制流关系，并将运行时步骤与塑造其行为的框架工件对齐。然后，它将失败归因于责任步骤和框架工件，并将反复出现的诊断整合为面向修复的缺陷记录。最后，HARNESSEXFIX 将这些记录映射到范围明确的修复算子，在缺陷特定的修复规范下生成补丁，并通过回归感知验证接受它们。我们在四个流行的基准上评估了 HARNESSEXFIX，结果表明它将初始框架的性能提高了 6.3% 至 18.4%，显著优于人工设计和自演化基线。HARNESSEXFIX 强调了将失败轨迹不仅视为反馈信号，而且视为用于诊断和修复智能体失败背后框架机制的结构化证据的价值。索引词——大语言模型智能体，框架工程，轨迹分析，失败归因，回归感知改进

执行、工具、上下文、生命周期、可观测性、验证与治理 [1]。例如，工具接口决定动作如何暴露和调用；上下文与记忆决定模型看到什么；生命周期与编排则控制执行流程和任务状态。当故障发生时，改进执行框架往往比修改基础模型更实用，因为基础模型的重新训练、微调或重新部署通常代价高昂。然而，执行框架的改进远非易事。与传统软件系统不同，传统系统中业务逻辑显式编码在控制流和数据结构中 [10]、[11]、[12]，智能体的行为在很大程度上由运行时模型推理决定，并受提示、检索到的上下文、工具接口和编排策略的影响 [1]、[13]、[14]。因此，故障通常无法直接映射到执行框架实现中的特定位置，如源代码或提示。相反，它们出现在交织了自然语言推理、工具交互、环境反馈和中间状态转换的执行轨迹中。这些轨迹以语言为主、由推理驱动且具有非确定性，使得传统的调试和修复技术效果显著降低 [15]、[16]、[17]、[18]、[19]。现有的自改进智能体和自动执行框架演化工作仅部分解决了这一问题。一类工作专注于运行时或监督式优化，利用轻量级监督和自适应纠正正在执行时绕过问题、暂时抑制错误，或在智能体周围添加外部补丁 [20]、[21]、[22]、[23]。这类策略虽然能改善观测性能，但往往未修复导致故障的底层执行框架缺陷。另一类工作主要以结果为导向而非以诊断为导向：它基于最终得分、成功率或效率指标来优化提示、工作流或执行框架逻辑 [7]、[24]、[25]、[26]、[27]，而并未首先识别轨迹中责任证据所在的位置以及哪个执行框架层发生了故障。缺乏这种有针对性的因果诊断，所产生的改进往往是宽泛的、间接的或范围界定不清的。在本文中，我们旨在从失败的智能体轨迹出发，详细诊断出负责的运行时步骤，并利用这些证据定位执行框架缺陷并进行修复。

I. 引言

大型语言模型智能体（LLM Agent）越来越多地用于仓库级软件工程、基于终端的流程、开放式研究和应用自动化中的多步骤、工具介导任务 [2]、[3]、[4]、[5]、[6]。与独立使用大型语言模型不同，这些智能体通过模型调用、工具调用、观察、中间工件、状态变化和最终提交，反复与外部环境交互。因此，此类系统的可靠性不仅取决于基础模型，还取决于智能体框架（Agent Harness）：即包裹模型并控制其行为的运行时基础设施 [7]、[8]、[9]。近期框架工程工作使用七层分类法描述该基础设施，包括

表 I: 执行框架层及其职责 [1]。

Harness layer	Responsibilities
Execution Environment and Sandbox	Provides safe, isolated, reproducible environments in which agent actions run with bounded autonomy.
Tool Interface	Governs how agents discover, describe, select, and invoke tools, including schemas, documentation, and error feedback.
Context and Memory	Determines what the model sees at each step: context window, session state, summaries, retrieved evidence, and persistent memory.
Lifecycle and Orchestration	Controls agent execution flow: think-act-observe loops, retries, task-state management, multi-agent coordination, and termination.
Observability	Records traces, logs, tool calls, errors, and cost information well enough to diagnose failures and monitor behavior.
Verification and Evaluation	Connects tasks to feedback through readiness checks, intermediate validation, final output evaluation, and regression testing.
Governance and Security	Defines permissions, identities, policies, approvals, and audit trails that constrain agent authority.

相应地。这一目标带来了若干关键挑战。首先，失败证据分散在语言密集型和推理驱动的轨迹中，涵盖多步交互、工具调用、自然语言推理和环境响应，这使得精确诊断变得困难。其次，即使能够在执行轨迹中定位失败，仍然难以将其映射回负责的具体测试框架实现，例如源代码、提示模板、工具规范、配置文件、适配器、插桩钩子和验证脚本。这种映射之所以困难，是因为执行轨迹捕获的是运行时行为，而测试框架实现定义在代码和提示等静态工件中；在智能体系统中，这两个视图通常不像传统软件那样明确对齐。第三，测试框架的修改可能会减少一种反复出现的失败，同时在其他地方引入回归。尽管这一挑战也出现在传统程序修复中，但在智能体系统中更为困难，因为运行时行为与测试框架机制之间的依赖关系不那么明确。

我们提出 HARNESSEXFIX，一个轨迹引导的框架，用于诊断智能体失败并修复智能体测试框架。它首先将原始执行轨迹和测试框架工件抽象为测试框架感知的轨迹中间表示（HTIR），该表示对步骤级运行时行为进行建模，重建数据流和控制流链接，并将轨迹与相应的测试框架实现工件对齐。HTIR 通过将碎片化的轨迹证据组织成适合归因的步骤级关系，为失败归因提供结构化的诊断基础，使 HARNESSEXFIX 能够识别负责失败的运行时步骤，解释根本原因，并将反复出现的诊断整合为缺陷记录。对于每个缺陷记录，HARNESSEXFIX 随后利用 HTIR 中捕获的运行时到实现的映射来定位修复目标，选择范围限定的修复算子，并实例化特定于缺陷的修复规范，以进行受约束的补丁生成和验证。

HARNESSEXFIX 所使用的修复算子基于我们对现实世界大语言模型智能体开发的经验研究。我们分析了 30 个流行的开源智能体仓库以及约 57,780 条开发记录。研究表明，与测试框架相关的变更在所有仓库中都 very 常见，并且测试框架缺陷横跨全部七项 ETCLOVG 职责，而非局限于提示词或单一组件。根据反复出现的缺陷模式和开发者的修复实践，我们总结了 HARNESSEXFIX 所使用的限定范围修复算子。这些发现为 HARNESSEXFIX 的设计提供了动机。

将测试框架修复视为多层运行时基础设施问题，并通过修复算子约束生成补丁的设计选择。我们在四个流行基准上评估了 HARNESSEXFIX：GAIA、SWE-Bench Verified、AppWorld 和 Terminal-Bench 2.0 Verified。在这些基准上，HARNESSEXFIX 相比初始测试框架将性能提升了 6.3% – 18.4%，并显著优于人工设计和自进化基线。消融结果进一步支持主要设计选择，表明基于迹的诊断和限定范围修复对最终收益至关重要。本文做出以下贡献：

- We present HARNESSEXFIX¹, a trace-grounded and 用于修复智能体框架的诊断驱动框架。与结果驱动的提示演化或自由形式的自我编辑不同，HARNESSEXFIX 明确地将失败证据与框架感知诊断和限定范围的框架更改联系起来。
- 我们进行了首个关于现实世界大语言模型智能体中框架缺陷的实证研究，揭示了跨框架层的缺陷分布，并总结了指导 HARNESSEXFIX 中限定范围修复的重复修复算子。
- 我们引入了 HTIR，一种框架感知的轨迹表示，将异构智能体轨迹规范化为步骤级证据结构，为步骤级归因、框架诊断和限定范围修复规划提供基础。
- 我们在四个流行的基准和智能体上评估了 HARNESSEXFIX，结果表明基于轨迹的框架修复将性能比初始框架提高了 6.3% – 18.4%，并显著优于人工设计的框架和自演化基线。

II. 背景与动机

A. 背景

智能体工程中的一个常见观点是，基于大语言模型的智能体由两部分组成：基础模型本身以及围绕它的其他一切，即提供工具、上下文、编排逻辑等的运行时基础设施。这些周边基础设施统称为智能体框架 [1]。当故障发生时，往往不仅源于模型的推理，还源于框架如何暴露信息、启用操作、记录证据和执行策略。

¹ 我们在匿名网站上发布了源代码 <https://github.com/HarnessFix/HarnessFix>。

框架通过可编辑的实现工件来实现，例如提示模板、工具规范、编排代码、配置文件、适配器、日志钩子和验证脚本。在本文中，框架修复指的是修改这些工件，而不是更新基础模型参数，以纠正诊断出的缺陷。为了描述构成智能体框架的机制，最近的工作引入了 ETCLOVG 分类法 [1]，它将框架职责组织为七个层次，如表 I 所示。在本文中，我们使用这些层次作为词汇表，将运行时证据与可能需要修复的框架机制联系起来。

B. 动机研究

为了理解框架缺陷在现实世界大语言模型智能体系统演化过程中如何产生和解决，我们对流行的开源智能体进行了动机研究。

1) **数据收集：**我们从 GitHub 收集了跨不同任务类别的流行且活跃的开源大语言模型智能体，得到 30 个大语言模型智能体系统和约 57,780 条开发记录，包括问题、拉取请求、提交和发布说明。我们使用基于大语言模型的分析智能体对开发记录进行分类，判断其是否与框架相关，并通过迭代过程进行人工验证，包括小规模示例标注、基于智能体的分类、人工检查、智能体策略优化以及最终抽样人工审计；由于篇幅限制，详细的收集和分类过程在我们的网站上提供。最终数据集包含 26,174 条与框架相关的记录。

总体而言，与测试框架相关的记录约占所有收集的开发记录的 45.3%，且全部 30 个代码仓库均包含此类变更，这表明开发者在实践中频繁遇到与测试框架相关的问题，并凸显了对自动化技术支持测试框架维护与修复的需求。此外，我们采用类似的大语言模型辅助且经人工验证的流程，来刻画测试框架缺陷的分布并总结常见的修复策略，详细过程因篇幅限制已发布在我们的网站上。

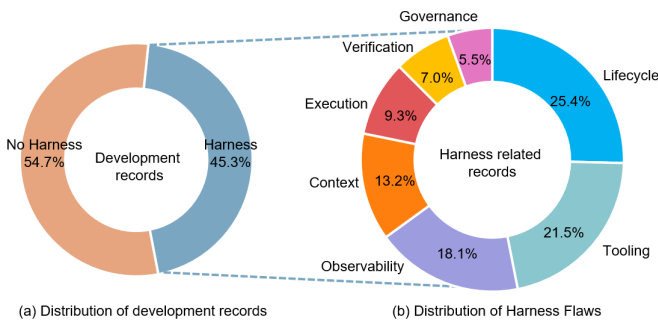


图 1：跨 ETCLOVG 层的分布。

2) **线束缺陷分布：**图 1 展示了七个治理层中治理缺陷的分布情况。缺陷出现在所有七个层中，其中生命周期、工具和

可观测性是缺陷最频繁的来源，数据集中几乎所有大语言模型智能体（30 个中有 29 个）都表现出与这三个层次相关的缺陷。我们还观察到，主导缺陷层次随仓库关注点而变化。生命周期缺陷在长时间运行或长视界智能体系统中更为常见 [28]、[29]、[30]、[31]、[32]、[33]、[34]，而验证缺陷在面向基准的测试框架和仓库中更常被观察到 [33]、[2]、[3]、[5]、[6]、[4]、[35]、[36]。这些结果表明，测试框架缺陷横跨运行时基础设施，而非仅限于提示或单一组件，这凸显了对能够处理多个测试框架层次的修复技术的需求，超越了现有技术所针对的提示优化或运行时上下文调整 [37]、[26]、[20]、[21]。3) **修复策略：**表二列出了我们为每个测试框架层次总结的典型修复策略。我们将这些策略组织为一组紧凑的修复算子，每个算子对应一类常见的测试框架修复操作。这些算子在第三-C 节中用于指导限定范围的测试框架修复。通过约束修复生成，它们降低了由自由形式修改技术对测试框架工件造成的不稳定编辑或广泛回归的风险 [38]、[39]、[40]、[41]。

三、方法

HARNESSFIX 的核心思想是从细粒度、基于迹的故障诊断出发驱动测试框架修复。它首先定位负责任的运行时步骤，将其行为映射到所涉及的测试框架缺陷，然后将这些诊断转化为范围明确的修复规范，以指导修复。HARNESSFIX 使用四个大语言模型智能体进行基于迹的测试框架修复。迹抽象智能体通过将运行时行为建模为迹步骤、重建数据流和控制流链接，并将得到的运行时证据与可编辑的测试框架工件对齐，从原始迹和测试框架工件构建测试框架感知的迹中间表示。诊断智能体消费该中间表示，将故障归因于负责任的迹步骤和所涉及的测试框架层，并将相似的诊断整合为缺陷记录，总结反复出现的根本原因。修复智能体将缺陷记录映射到修复算子，实例化缺陷特定的修复规范，并据此生成范围明确的候选补丁。验证智能体检查实现是否保持在预期的修复范围内，并在不引入不可接受的回归的情况下减少目标缺陷。在介绍 HARNESSFIX 之前，我们首先展示一个来自实验基准 AppWorld 的运行示例，如图 3 所示，本节其余部分将使用该示例来说明该方法。

A. 建模运行时轨迹并将其与

测试框架实现对齐 原始轨迹记录运行时行为，但测试框架修复既需要识别导致故障的运行时证据，也需要定位实现或支配该证据的可编辑测试框架工件。因此，HARNESSFIX 首先

表 II：按测试框架层组织的修复算子及相关缺陷

Harness layer	Typical flaws	Candidate repair operators
Execution Environment and Sandbox	Actions run in environments that are non-reproducible, insufficiently isolated, state-leaking, or blocked by interactive prompts.	Sandbox-boundary tightening; environment snapshot/restore; file-system/network/API access isolation; task-specific sandboxing; environment and state-diff logging.
Tool Interface	Tool schemas are ambiguous, candidate tool sets are poorly ranked, parameters are weakly validated, or tool errors are not actionable.	Tool-schema narrowing; argument validation; tool-candidate ranking and retrieval; tool documentation and error-message repair; read/write tool separation; transactional tool wrapping.
Context and Memory	Critical evidence, constraints, state, or prior results are omitted, stale, polluted, overly long, or fragmented.	Failure-relevant evidence preservation; artifact/state summary exposure; task-constraint refresh; summary and retrieval policy repair; persistent-memory isolation; context-budget checking.
Lifecycle and Orchestration	The control flow enters retry loops, repeats ineffective actions, finalizes prematurely, loses task state, or delegates without verification.	Loop guarding; explicit task-state modeling; retry and timeout bounding; verification-gated finalization; workflow-state checkpointing; delegated-output validation.
Observability	Traces lack the fields needed to diagnose failures or connect outcomes to requests, tool calls, errors, and state changes.	Model-request instrumentation; context-snapshot logging; tool-call/result tracing; API-error and state-delta logging; retry/token/latency tracking; structured trace logging.
Verification and Evaluation	Readiness checks, validators, judges, or regression tests do not catch invalid artifacts, missing effects, or task-inconsistent states.	Pre-execution readiness checking; intermediate validation gating; expected/actual state comparison; effect-evidence completion guarding; finalization-check strengthening; regression testing.
Governance and Security	Permissions, policy checks, approval flows, identity boundaries, and audit trails are too weak for high-impact actions.	Least-privilege credentialing; high-impact action approval gating; policy-check enforcement; audit-decision logging; out-of-scope action blocking; escalation-rule definition.

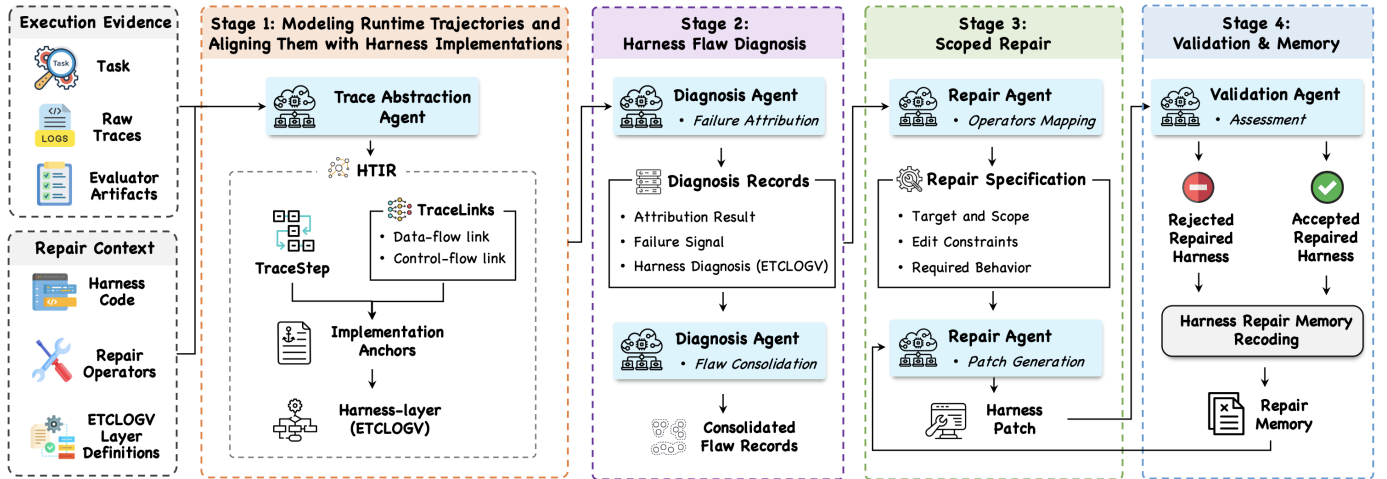


图 2：HARNESSEXFIX 概览。

构建测试框架感知的迹中间表示，该表示有两个目的。首先，它将轨迹建模为迹步骤，并重建跨步骤的迹链接，包括数据流和控制流链接，为步骤级故障归因提供所需证据。其次，它通过实现锚点将迹步骤和迹链接与测试框架实现对齐，使负责任的运行时证据能够映射到具体工件。该建模与对齐阶段为识别负责任的运行时步骤、诊断所涉及的 ETCLOV 层以及约束后续测试框架修复提供了基础。

1) 迹步骤：运行时证据单元： 分层迹信息表示中的每个节点代表迹中一个可恢复的执行步骤，例如模型调用、工具中介动作或最终提交决策。我们将每个节点称为迹步骤。

每个迹步骤被赋予唯一标识符，该标识符为顺序整数，指示其在执行顺序中的位置。它还存储两个基本字段，即请求消息和响应消息。它们分别是该步骤发送给模型、工具或环境或由其返回的完整消息。分层迹信息表示还为每个迹步骤添加三个派生标注。

(a) 角色是步骤的功能，例如信息获取、工具调用、工件编辑、验证、编排决策或最终提交。(b) 执行状态包括成功、失败、超时、阻塞等。(c) 工件/状态效应记录该步骤产生的外部可观察效应的类型，以及相应的效应细节。其效应类型指示该步骤是否没有可观察的外部效应、只读交互、工件变更、环境或应用状态变更、混合效应或未知效应。

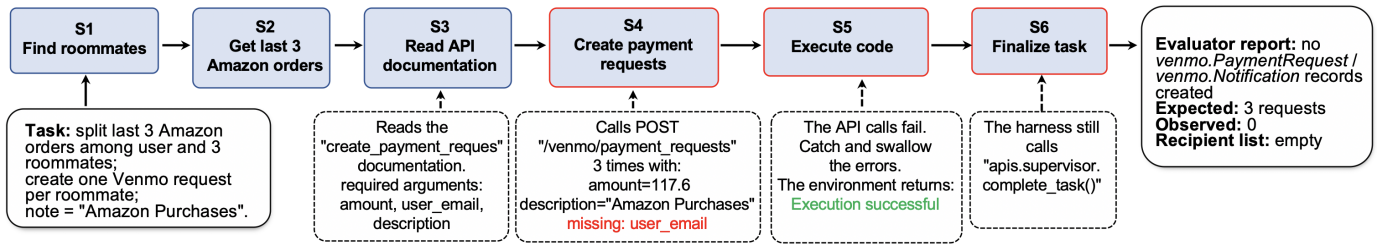
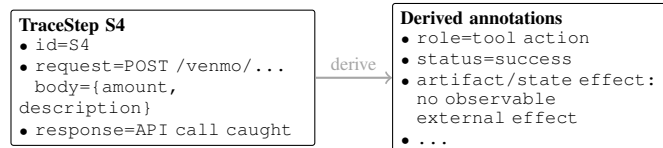


图 3：应用世界完成守卫测试框架缺陷的说明性示例。

迹抽象代理检查每个迹步骤的请求和响应消息以推断这三个标注。对于工件/状态效应，代理检查消息以及任何可用的工具或环境元数据，以识别外部可观察的后果。效应细节在可用时指定三类信息：a) 受影响的工件或状态实体，例如文件、数据库记录或会话状态；b) 观察到的状态转换，例如文件创建、数据库更新或会话状态变更；c) 支持证据，例如工具返回值、执行日志或状态差异快照。以下示例展示了一个迹步骤及其请求、响应和派生标注。



2) 数据流对齐：将证据传播

与测试框架逻辑关联：HARNESSFIX 构建数据流链接，以解释信息如何通过先前的轨迹内容和请求构造逻辑，在后续面向模型的请求中被输入、消失或转换。这种对齐至关重要，因为许多测试框架缺陷源于关键证据、约束、工具输出、错误消息或状态变化在后续步骤中丢失、过时、污染、错误总结或未被消费。为了推导数据流链接，迹抽象代理联合检查当前请求消息、先前的迹步骤以及组装面向模型输入的测试框架工件。它以逆时间顺序搜索先前的迹步骤，并识别显式重用（例如复制或拼接的消息片段）和语义重用（即当前请求基于先前的观察，尽管表面措辞不同）。它还检查请求构造逻辑，包括内存组装、工具描述、提示模板和其他提示构建组件，以确定证据是如何被插入、总结、过滤或省略的。每个数据流链接记录源迹步骤标识符、目标迹步骤标识符、先前请求或响应中的源片段、当前请求中的目标片段以及重用关系（例如复制、总结或语义重用）。片段是原始迹记录的精确切片，可验证地支撑该链接，从而实现具体的证据归因。

下图展示了一个数据流链接的示例。



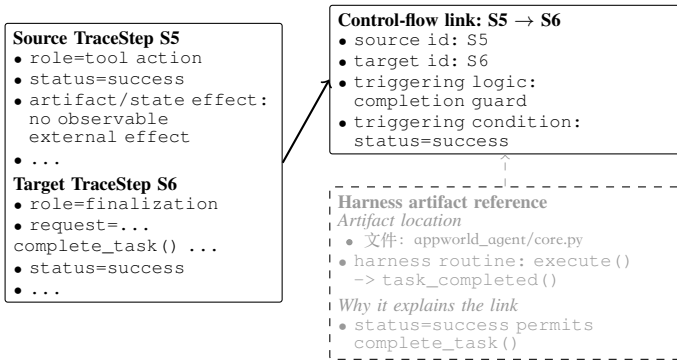
S3 应用程序编程接口文档中关于 create_payment_request 的内容（包括其必需参数和请求模式）与 S4 支付请求体存在语义链接。该链接使得缺失的必需 user_email 字段变得可观察，从而支持后续的故障诊断。

3) 控制流对齐：解释测试框架驱动的

执行决策：HARNESSFIX 构建控制流链接，以解释测试框架为何从一个跟踪步骤（TraceStep）过渡到另一个。数据流与控制流链接互为补充，因为智能体故障可能源于跨步骤的信息传播或步骤过渡逻辑。

为推导控制流链接，迹抽象代理检查当前迹步骤、其时间邻域中的前后迹步骤、关联的数据流链接以及相关的测试框架实现。随后，它将观察到的转换与测试框架执行逻辑（如继续、重试、委托、验证、终结或终止）进行匹配，并在有证据可用时记录触发当前迹步骤的条件或执行状态。每个控制流链接记录源迹步骤标识、目标迹步骤标识、触发逻辑，以及可用的触发条件或执行状态。

下图展示了一个控制流链接的示例。尽管 S5 返回成功，但其工件/状态效应没有可观测的外部效果，表明预期的状态变化并未发生。完成守卫错误地将此执行状态（即成功）视为任务进展的证据，并允许 S6（一个终结追踪步骤）调用 complete_task()。因此，该链接捕捉了导致错误终结的控制器决策。



4) 实现锚点：定位运行时证据

线束工件中的密度：迹步骤（TraceSteps）及其关联的数据流和控制流链接描述了运行时事件和跨步骤关系，但它们本身并不指明相应行为在测试框架（harness）实现中的具体位置。因此，分层迹中间表示（HTIR）将运行时证据与实现锚点关联起来，这些锚点定位了实现或支配所观察行为的可编辑测试框架工件。迹抽象智能体通过联合检查轨迹和测试框架工件来推导实现锚点。它利用可用的运行时信号，如工具名称、提示模板引用、适配器调用、验证器事件和控制器动作，来定位相应的实现工件。当某个关系跨多个组件实现时，智能体遵循先前构建的控制流和数据流链接，以识别实现或约束该关系的测试框架工件。每个锚点记录三个字段：工件引用，例如仓库路径和行范围、提示或配置标识符、工具模式标识符、工作流节点标识符或验证器标识符；锚点关系，例如发出或执行迹步骤、构造步骤输入或支配控制流转换；以及支持此对齐的证据。下面的示例将 S3 - S4 数据流链接（如前所示）落实到暴露 S3 应用程序编程接口（API）文档的提示渲染工件中。

Implementation anchor

- 工件引用: prompts/react_code_agent/instructions.txt
code/simplified/react_code_agent.py
SimplifiedReActCodeAgent.initialize()
- anchor relation: constructs S4 input
- Relevant prompt/template lines
instructions.txt:
"Interact with app(s) ... using APIs"
show_api_doc(app_name, api_name)
"Each code execution will produce
an output ..."; Task: {{ instruction }}
- initialize():
template.render(instruction, main_user,
app_descriptions) -> self.messages
- supporting evidence: ...

5) 测试框架层职责映射：在迹步骤及其关联链接通过实现锚点连接到测试框架工件之后，分层迹中间表示（HTIR）将每个迹步骤周围的锚定证据映射到 ETCLOVG 职责层。此映射用于整合重复出现的

缺陷记录、选择限定范围的修复算子，并分析不同基准测试失败所牵涉的测试装置职责。这是迹特定的，而非静态的文件级分类：迹抽象代理以迹步骤的信息、关联链接和实现锚点为输入，将其与表一中各层的职责准则进行比较。当锚定的运行时证据涉及多个测试装置职责时，一个迹步骤可以映射到多个层。利用上述诊断信息，我们分析 S6 的层映射。S6 在 S5 - S6 转换由 status=success. 启用后调用 complete_task()。支持性的 S3 - S4 数据流链接提供了上下文诊断证据：S3 应用程序编程接口文档指定了必需的 user_email 字段，但 S4 支付请求体省略了该字段。综合这些信号表明，尽管存在未解决的上游请求构造错误以及缺少预期的工件/状态效果，S6 仍完成了任务，导致其映射到生命周期层和验证层。本讨论聚焦于 S6 的层映射；该案例的完整缺陷记录还涉及 S5 可观测试证据，表明应用程序编程接口错误被吞掉，且工件/状态效果证据未暴露给测试装置。

B. 测试装置缺陷诊断

1) 失败归因：给定一个失败的执行迹，失败归因识别哪些迹步骤对失败负责，确定根本原因，并将其关联到特定的测试装置层。

诊断智能体通过四个步骤执行任务。首先，症状定位通过检查外部评估结果以及最终迹步骤的诊断证据来识别失败之处，包括迹步骤详情、关联链接、实现锚点和映射的测试框架层。其次，证据回溯沿着从最终迹步骤到更早迹步骤的数据流和控制流链接进行，并生成一组排序的候选责任迹步骤。此阶段有意保持宽泛：候选步骤之所以可疑，是因为它可能通过数据传播、控制转移、工件或状态效应、或锚定的测试框架机制与观察到的失败存在合理关联。第三，候选裁决检查每个候选步骤的诊断证据，以判断该步骤是否形成、传播、未能暴露、未能约束或未能验证与失败相关的信息或状态变化。然后选择最终的责任迹步骤。第四，层分配将失败归因于已映射到最终责任迹步骤的测试框架层。对于每次失败的迹，HARNESSFIX 生成一个结构化的诊断记录，包含三个部分：（a）归因结果标识责任迹步骤并提供简洁的根本原因解释。（b）失败信号记录观察到的失败症状以及支持该归因的相关诊断证据。（c）测试框架诊断包括一个或多个涉及的 ETCLOVG 测试框架层以及对测试框架缺陷的简洁描述。

2) 测试框架缺陷整合：单次失败执行可能是偶然的。因此，HARNESSFIX 在修改测试框架之前，会整合跨任务执行的诊断记录。诊断智能体首先根据涉及的测试框架层对诊断记录建立索引，并将具有重叠层的记录视为合并候选。然后合并那些根本原因解释和支持证据表明存在相同重复性测试框架缺陷的候选记录。

每个生成的缺陷记录代表一种反复出现的测试装置缺陷模式，并组织为两组字段：（a）缺陷摘要描述反复出现的测试装置缺陷。它包括缺陷标识符、一个或多个涉及的测试装置层，以及共同的根本原因摘要。（b）支持诊断记录总结该缺陷的经验依据。它包括代表性诊断记录，以及用于合并它们的共享诊断依据，例如跨分组诊断记录的常见故障信号、负责的跟踪步骤和根本原因理由。

C. 限定范围修复

在缺陷合并之后，HARNESSFIX 必须决定对每个反复出现的缺陷进行何种测试装置更改。此阶段有意加以约束，因为故障归因所涉及的测试装置工件通常属于核心运行时逻辑，例如编排和上下文构建。允许修复代理自由修改此类工件，如自改进编码代理系统 [38]、[39]、[40]、[41] 中那样，将带来不稳定编辑、架构破坏或广泛回归的高风险。因此，HARNESSFIX 应用修复算子，这些算子源自我们的动机研究并总结于表 II，作为限定范围的修复操作，规定针对已诊断缺陷可以更改哪些测试装置机制。HARNESSFIX 首先将每个缺陷记录映射到适用的修复算子，并利用缺陷记录的支持诊断记录实例化所选算子，以推导出缺陷特定的修复规范。

1) **将缺陷映射到修复算子：**该阶段的输入是一条缺陷记录，HARNESSFIX 将其映射到可应用的修复算子。修复代理首先利用缺陷记录中涉及的线束层，从表 II 中检索候选算子组。然后，它根据缺陷记录的常见根本原因和诊断证据对这些候选算子进行过滤。从选定的算子中，它指定一个主算子，依据是其可编辑机制直接针对所涉及的 TraceStep 的程度，以及其预期效果是否直接满足根本原因所隐含的行为需求。其余选定的算子作为辅助算子：它们暴露证据、强制执行约束，或修改主修复所需的相邻线束机制。选定的算子及其选择理由被带入修复规范中，使补丁验证能够检查生成的差异是否遵循预期的修复范围。2) **基于修复规范的补丁生成：**在为缺陷记录选定修复算子后，HARNESSFIX 将它们实例化为候选修复的修复规范。

实现。该规范的目的是将操作符级别的修复意图转化为针对特定缺陷的实现契约，即，它将操作符绑定到当前的缺陷记录和目标测试装置。由此产生的规范用针对特定缺陷和特定测试装置的约束替换了抽象的操作符字段。每个规范包含三组字段。

（a）目标和范围将规范绑定到具体的缺陷记录、其涉及的测试装置层、选定的修复操作符以及代表性的诊断记录。（b）编辑约束将操作符的可编辑资源模板绑定到目标实现中的具体测试装置工件，同时列出禁止的工件。（c）所需行为将操作符的预期效果转化为具体的行为要求。它描述了在应用候选更改后必须保持的测试装置行为。下面的示例概述了由图 3 中任务的合并缺陷记录实例化的修复规范。

Repair specification

- 目标与范围：
 - 缺陷 record=completion 在无工件/状态影响下被接受
 - 涉及 layers=Lifecycle,
- Verification, Observability
 - selected operators=verification-gated finalization; effect-evidence completion guarding; API-error and state-delta logging
 - representative diagnoses=...
- 编辑约束：
 - 允许 artifacts=appworld_agent/core.py; 执行结果插桩
 - 禁止 artifacts=task 数据/预言机、验证/测试标签、公共应用程序编程接口
- 所需行为：
 - 暴露应用程序编程接口错误证据
 - 在 complete_task() 之前要求工件/状态效果证据

e.g., PaymentRequest records

D. 补丁验证与工具记忆

在修复规范下生成候选补丁后，HARNESSFIX 进入验证阶段，以确定该补丁是否应被纳入工具。验证代理首先对候选差异（生成的补丁与原始实现之间的差异）执行预验证检查，以确定其是否符合修复规范并满足基本的实现正确性，例如语法和静态分析检查。然后，它通过在留出的验证集上评估候选补丁来进行验证集评估，检查它是否减轻了目标缺陷，同时避免在原始工具先前解决的任务上出现不可接受的回归。仅当补丁达到目标改进并保持在回归限制内时，才被接受。HARNESSFIX 将结果记录为工具修复记忆。每条记忆记录包含缺陷记录、涉及的工具层、修复规范、差异摘要、预验证检查和验证集评估的结果、发生回归的验证任务，以及关于修复适用或不适用任务条件的自然语言描述。被接受的记录为未来类似的缺陷记录提供实现

示例。被拒绝的记录解释原因是预验证失败、目标改进不足还是回归过度，并有助于防止未来迭代提出相同的无效更改。

IV. 实验设计

A. 研究问题

RQ1：有效性与效率。HARNESSFIX 是否实现了更好的任务性能和令牌效率？

RQ2：故障诊断。HARNESSFIX 能否产生有效的故障诊断？

研究问题三：消融实验。HARNESSFIX 的设计选择是否均有助于性能提升？

研究问题四：跨模型迁移。从某一任务模型导出的框架修复能否迁移至其他任务模型？

B. 智能体与基准

我们在四个具有代表性的智能体框架基准上评估 HARNESSFIX，涵盖开放式研究问答、仓库级软件修复、有状态应用自动化以及基于终端的命令行工作流。对于每个基准，我们选择一个初始框架 H_0 作为 HARNESSFIX 和自进化基线的起点，并将采样任务随机划分为训练集、验证集和留出测试集。对于 GAIA [5]，我们使用开放式深度研究框架 [32] 作为 H_0 ，并从 166 个带有公开真实答案的标注开发者问题中采样 150 个，按 60/30/60 划分。对于 SWE-Bench Verified [3]，我们使用 mini-swe-agent [42] 作为 H_0 ，并从 500 个实例中采样 250 个，按 100/50/100 划分。对于 AppWorld [6]，我们使用官方简化版 ReAct 代码智能体作为 H_0 ，并从 750 个任务中采样 225 个，按 90/45/90 划分。对于 Terminal-Bench 2.0 Verified [4]，我们使用 Harbor Terminus-2 终端智能体框架 [34] 作为 H_0 ，并从 89 个任务中采样 85 个，按 34/17/34 划分。遵循标准基准实践，我们以任务完成率（简称 TCR）衡量性能。

C. 基线

研究问题一将 HARNESSFIX 与两类基线进行比较：人工设计的测试框架基线和自进化与修复基线。

人工设计的测试框架基线。对于每个基准，我们与代表性的人工设计测试框架进行比较：GAIA 上的 DeepResearchAgent [31] 和 MiroFlow [30]；SWE-Bench Verified 上的 OpenHands [43] 和 Trae-Agent [44]；AppWorld 上的 FullCodeRef1、IPFunCall 和 CUGA [45]；以及 Terminal-Bench 2.0 Verified 上的 OpenCode [46] 和 OpenHands [43]。

自进化与修复基线。我们还与四种从观察到的执行中调整智能体行为的代表性方法进行比较：1) GEPA [37]，一种具有帕累托前沿选择的反思性提示进化方法；2) SCOPE [26]，一种记忆引导的提示进化方法；

3) ReCreate [8]，它将轨迹转换为可复用的

任务领域脚手架和工作流指导；以及 4) Meta-Harness [47]，它根据先前的测试框架版本、任务得分和执行迹来优化测试框架代码。对于这些基线，我们还记录了令牌消耗量（以百万计），简称为 Tokens (M)。

D. 实验设置

我们使用 GPT-5 mini 作为运行实验智能体、我们的 HARNESSFIX 以及基线的默认模型。我们还实验了另外四种大语言模型（LLM）进行比较，并评估跨模型迁移性能（见研究问题 4）。详细的模型配置见我们的网站。表四、表六、表三和表七中的任务性能值是三次独立运行的算术平均值。为了回答研究问题 2，我们从测试集上的测试框架失败中构建了一个人工标注的诊断集，该信息不用于测试框架修复或验证决策。对于每个基准，三位标注者为 20 条失败的测试轨迹标注了负责的失败步骤、根本原因、实现锚点、涉及的测试框架层以及选定的修复算子。这些标签作为评估的金标准，我们报告了各标注维度上的诊断准确率。完整的标注和度量细节见我们的网站。为了回答研究问题 3，我们评估了四种消融设置，它们移除或削弱了 HARNESSFIX 中的关键设计选择：1) 仅提示修复，仅应用提示更新并禁用运行时测试框架更改；2) 无轨迹锚定诊断，移除测试框架缺陷诊断（第 III-B 节），并从原始轨迹摘要中推导修复上下文；3) 无范围修复算子，用自由形式的测试框架编辑替换范围修复（第 III-C 节）；以及 4) w/o 回归感知验收，移除针对目标缺陷减少和新引入回归的验证集验收检查（第 III-D 节）。为了回答研究问题 4，我们使用通过 GPT-5 mini 选择的修复后的 GAIA 测试框架，并在相同的留出 GAIA 测试分割上直接复用于四个额外的任务模型：Qwen3.5 Plus、DeepSeek V3.2、Gemini 3 Pro 和 Claude Sonnet 4.5。除了执行目标应用程序编程接口所需的最小模型提供商兼容性处理外，这些迁移运行不执行特定于目标模型的测试框架修复。

五、结果与分析

A. 有效性及效率（研究问题 1）

表三在四个基准和五个大语言模型上比较了 HARNESSFIX 与初始测试框架 H_0 。HARNESSFIX 在所有设置中均一致优于 H_0 ，平均提升 11.1%。最大的改进出现在 GAIA 上，范围从 14.5% 到 18.4%，而即使在 AppWorld 上的最小增益也保持在 5.9% 到 9.3% 之间。不同大语言模型的绝对分数有所不同，但引入的改进相对模型一致：每个大语言模型的平均增益范围从 10.3% 到 12.5%。

表三：在 H_0 (RQ1). 上的性能

Bench.	LLM used	H_0	H_0 +HARNESSFIX	Δ
GAIA	GPT-5 mini	43.3	61.7	+18.4
	Claude Sonnet 4.5	66.7	84.4	+17.7
	DeepSeek V3.2	58.9	75.6	+16.7
	Qwen3.5 Plus	63.3	78.9	+15.6
	Gemini 3 Pro	69.4	83.9	+14.5
SWE	GPT-5 mini	45.3	57.3	+12.0
	Claude Sonnet 4.5	61.3	71.7	+10.4
	DeepSeek V3.2	58.7	69.3	+10.6
	Qwen3.5 Plus	62.7	72.3	+9.6
	Gemini 3 Pro	60.3	68.7	+8.4
App.	GPT-5 mini	36.7	43.0	+6.3
	Claude Sonnet 4.5	61.1	70.4	+9.3
	DeepSeek V3.2	56.3	62.2	+5.9
	Qwen3.5 Plus	53.3	61.9	+8.6
	Gemini 3 Pro	50.7	59.6	+8.9
TB2	GPT-5 mini	17.6	26.5	+8.9
	Claude Sonnet 4.5	34.3	47.1	+12.7
	DeepSeek V3.2	31.4	39.2	+7.9
	Qwen3.5 Plus	42.2	52.9	+10.7
	Gemini 3 Pro	46.1	55.9	+9.8

表四：性能比较（研究问题一）。

Bench.	Role	Method / harness	TCR	Tokens (M)
GAIA	Selected H_0	open-deep-research	43.3 (+18.4)	/
	Human-designed	DeepResearchAgent	52.8 (+8.9)	/
		MiroFlow	58.3 (+3.4)	/
	Evolve/repair from H_0	H_0 + GEPA	46.7 (+15.0)	32.6 (-43.4)
		H_0 + SCOPE	45.0 (+16.7)	26.9 (-53.3)
		H_0 + ReCreate	52.8 (+8.9)	67.9 (+17.9)
		H_0 + Meta-Harness	56.7 (+5.0)	94.2 (+63.5)
H_0 + HARNESSFIX		61.7	57.6	
SWE	Selected H_0	mini-swe-agent	45.3 (+12.0)	/
	Human-designed	OpenHands	47.3 (+10.0)	/
		Trae-Agent	48.7 (+8.6)	/
	Evolve/repair from H_0	H_0 + GEPA	46.7 (+10.6)	28.1 (-42.5)
		H_0 + SCOPE	48.3 (+9.0)	25.6 (-47.6)
		H_0 + ReCreate	51.7 (+5.6)	57.2 (+17.0)
		H_0 + Meta-Harness	54.7 (+2.6)	82.9 (+69.5)
H_0 + HARNESSFIX		57.3	48.9	
App.	Selected H_0	ReAct	36.7 (+6.3)	/
	Human-designed	FullCodeRefl	35.6 (+7.4)	/
		IPFunCall	38.1 (+4.9)	/
		CUGA	41.1 (+1.9)	/
	Evolve/repair from H_0	H_0 + GEPA	37.4 (+5.6)	22.7 (-39.0)
		H_0 + SCOPE	38.9 (+4.1)	18.8 (-49.5)
		H_0 + ReCreate	39.3 (+3.7)	44.1 (+18.5)
H_0 + Meta-Harness		40.4 (+2.6)	74.6 (+100.5)	
H_0 + HARNESSFIX		43.0	37.2	
TB2	Selected H_0	Harbor Terminus-2	17.6 (+8.9)	/
	Human-designed	OpenCode	22.5 (+4.0)	/
		OpenHands	18.6 (+7.9)	/
	Evolve/repair from H_0	H_0 + GEPA	19.6 (+6.9)	18.6 (-44.5)
		H_0 + SCOPE	20.6 (+5.9)	18.1 (-46.0)
		H_0 + ReCreate	21.6 (+4.9)	38.8 (+15.8)
		H_0 + Meta-Harness	23.5 (+3.0)	66.8 (+99.4)
H_0 + HARNESSFIX		26.5	33.5	

注：在 TCR 中，括号内显示 HARNESSFIX 相对于该行的百分点增益。在 Tokens 中，括号内显示相对于 HARNESSFIX 的相对 token 变化；/ 表示不适用。

在表四中，我们将骨干模型固定为 GPT-5 mini（上述评估的五种模型之一），并将 HARNESSFIX 与所有基线进行端到端比较。HARNESSFIX 优于

表五：失败诊断评估（研究问题二）。

Input representation	Step	Cause	Anchor	Layer	Operator
Raw trace	55.0	53.8	50.0	58.4	51.3
Raw + data-flow	70.0	68.8	65.0	73.2	66.3
Raw + data/control	77.5	75.0	72.5	79.4	73.8
Full HTIR	85.0	83.8	81.3	86.2	82.5

表六：消融实验（研究问题三）。

Variant	GAIA	SWE	AppWorld	TB2
H_0	43.3	45.3	36.7	17.6
Prompt-only repair	50.6	48.3	37.4	18.6
w/o trace-grounded diagnosis	51.1	50.7	38.1	21.6
w/o scoped repair operators	50.6	49.3	37.4	18.6
w/o regression-aware acceptance	55.6	53.3	39.3	24.5
Full HARNESSFIX	61.7	57.3	43.0	26.5

人工设计的 harness 平均高出 6.3%，提升幅度在 1.9% 至 10.0% 之间，且无需人工设计 harness 的工作量。在从相同的 H_0 , H ARNESSFIX 初始化的自动化自我进化和修复基线中，HARNESSFIX 取得了最佳性能，平均比这些基线高出 6.9%，增益范围从 2.6% 到 16.7%。即使与最强的自动化基线 Meta-Harness 相比，HARNESSFIX 仍高出 2.6% 至 5.0%，同时使用的 token 数量显著减少。具体而言，Meta-Harness 消耗的离线进化/修复 token 比 HARNESSFIX 多 63.5% 至 100.5%，表明我们的方法同时提升了有效性和 token 效率。为评估这些改进的统计显著性，我们使用所有基准上的重复运行结果进行单侧配对符号检验。对于表四中的基线比较， p 值范围从 2.4×10^{-4} 到 4.9×10^{-4} ，均低于 0.001，表明 HARNESSFIX 的改进具有统计显著性。

B. 失败诊断（研究问题二）

表 V 显示，完整的 HTIR 与人工标注的金标准标签高度一致，步骤准确率达到 85.0%，实现锚点准确率为 81.3%，测试框架层宏 F1 值为 86.2%，修复算子准确率为 82.5%。对于细粒度的步骤级定位，这一结果显著优于现有技术通常达到的约 45 – 52% 的步骤准确率 [48]、[49]，将诊断提升到足以支持针对性修复而非盲目优化的水平。与原始轨迹相比，加入数据流和控制流链接一致地改善了诊断指标。这些结果表明，HARNESSFIX 的收益源于结构化的、基于轨迹的诊断。

消融实验（RQ3）

如表 VI 所示，所有消融均降低了相对于完整 HARNESSFIX 的性能，表明每个设计选择都对最终结果有所贡献。当修复仅限于提示更新或移除作用域修复算子时，性能下降最为显著，这表明有效的测试框架修复依赖于对运行时机制的有针对性更改。

表 VII: GAIA 上的跨模型迁移 (RQ4)

Model	H_0	$H_0 + \text{HARNESFIX}$	Δ
GPT-5 mini (repair source)	43.3	61.7	+18.4
Claude Sonnet 4.5	66.7	72.2	+5.5
DeepSeek V3.2	58.9	66.7	+7.8
Qwen3.5 Plus	63.3	72.8	+9.5
Gemini 3 Pro	69.4	78.3	+8.9

表 VIII: HARNESFIX 与基线修复的测试框架层。=主要编辑目标，● =部分或间接编辑目标，○ =未编辑。

Layer	HARNESFIX repairs				Baselines			
	GAIA	SWE	App.	TB2	GEPA	SCOPE	ReC.	Meta-H.
Execution	●	●	○	○	○	○	○	●
Tool Interface	●	●	○	○	○	○	○	●
Context/Memory	○	●	●	●	●	●	●	●
Lifecycle	●	●	○	●	○	○	○	●
Observability	●	○	○	○	○	○	○	○
Verification	○	●	●	●	○	○	○	●
Governance	○	●	○	●	○	○	○	○
# layers touched	5	7	5	4	1	1	2	7

Baseline columns summarize method-level edit scope. ReC. = ReCreate; Meta-H. = Meta-Harness.

跨模型迁移 (RQ4)

表 VII 报告了使用 GPT-5 mini 修复的 GAIA 测试框架是否能在不进行目标特定轨迹分析的情况下迁移到其他模型。修复后的测试框架使修复源模型性能提升了 18.4%，同时也提升了所有四个目标模型的性能，带来 5.5% 至 9.5% 的一致迁移增益。这些增益小于源模型增益，但始终为正，表明修复解决了跨模型共享的测试框架层故障。

讨论

表 VIII 显示，大多数基线仅修改测试框架的狭窄部分：GEPA 和 SCOPE 触及一层，ReCreate 触及两层，使其不太适合跨越多个测试框架职责的故障。Meta-Harness 可以触及全部七层，但其基于优化的搜索缺乏针对性，因此如表 IV 所示产生高得多的令牌成本。相比之下，HARNESFIX 利用基于迹的定位和诊断驱动的框架执行有范围的编辑，无需穷举搜索即可覆盖多样的测试框架缺陷。

表 VIII 还展示了 HARNESFIX 执行的具体测试框架修复。在开放式研究问答 (GAIA) 中，缺陷通常集中在工具接口和可观测性上，例如缺少应用程序编程接口配置、不支持的文档格式、脆弱的媒体转换，以及未能后续答案综合保留足够证据的迹。在仓库级软件修复 (SWE-Bench Verified) 中，许多故障涉及工具执行、上下文构建和验证证据之间的接口：智能体必须发出 shell 命令和文件编辑，观察正确的测试输出，并在基准特定约束下提交补丁。在有状态应用自动化 (AppWorld) 中，任务需要应用程序编程接口调用或生成的代码来产生持久

应用状态效应则呈现出不同的模式。其失败往往同时涉及工具接口、生命周期、验证与治理等方面，因为测试框架必须验证应用程序编程接口参数、暴露应用程序编程接口错误、跟踪工件/状态效应，并避免在所需副作用发生之前就接受任务完成。终端式命令行任务 (终端基准 2.0 已验证) 对上下文与记忆、生命周期、验证与治理提出了更高要求：测试框架必须保留终端上下文、管理命令行工作流状态、优先采用基准提供的测试而非临时检查，并避免无效的终端操作。这些模式表明，测试框架修复不仅仅是一个提示优化问题。不同的基准需要不同的运行时机制，而一次修复往往具有跨层启示。例如，保留部分工作的生命周期修复也可能改变验证可用的证据以及应接受最终化的条件。由于 HARNESFIX 的诊断围绕每个反复出现的缺陷聚合所涉及的层次，其修复规范和验证检查可以在接受候选测试框架变更之前考虑这些耦合效应。

七、相关工作

智能体测试框架。近期研究将智能体测试框架视为一等工程对象，而非附带的胶水代码。综述与实践导向的工作刻画了测试框架的职责与运行时表面 [1]、[50]、[28]。另一条研究脉络关注测试框架的规范与组合：例如自然语言智能体测试框架 [51] 和智能体流程 [52]。第三条脉络自动化测试框架的优化或演化，包括自动测试框架 [53]、元测试框架 [47] 和智能体测试框架工程 [54]。HARNESFIX 基于这一观点，但利用失败轨迹进行诊断性修复。

自改进智能体。自我改进智能体的工作跨越多个修复层面。智能体设计与工作流搜索方法优化智能体结构、工作流或多智能体架构 [55]、[56]、[57]、[58]、[59]。工具与经验导向的方法将先前的交互提炼为可复用的工具、技能、工作流、记忆、提示或领域智能体 [60]、[61]、[62]、[27]、[63]、[8]、[64]、[65]、[66]、[67]、[68]、[37]。自我修改与进化方法递归地修改智能体代码、决策模块、协作网络或开发协议 [38]、[39]、[40]、[41]、[69]、[70]、[71]、[72]。验证驱动的方法使用基于评分标准的检查或测试时学习来改进推理时的行为 [73]、[72]。相比之下，HARNESFIX 从失败的轨迹中修复框架：它将失败归因于负责的 TraceStep 和框架层，然后应用范围限定的框架更改，而不是演化整个智能体、仅仅重用经验或仅加强推理时的验证。轨迹分析、失败归因与智能体操作。

Wang 等人 [74] 进行了首个关于大语言模型智能体轨迹分析的综述。一些方法专注于表示或重构轨迹，例如 SWE-TRACE [75] 和 VCC [76]。其他方法分析失败如何随时间出现 [77]、[29]。FAMAS [78]、AgenTracer [79]、AgentFixer [80]

和 DoVer [49] 进一步研究失败归因与根因诊断。这些工作展示了轨迹级诊断的价值；HARNESSFIX 进一步通过利用诊断来实施框架更改，从而闭合循环。

VIII. 结论

随着基于大语言模型的智能体被应用于日益复杂的任务，基础模型周围的框架成为系统可靠性的重要组成部分。本文提出 HARNESSFIX，它构建 HTIR 将异构轨迹和框架工件组织为步骤级证据，将失败归因于负责的步骤和框架层，并在缺陷特定的修复规范指导下生成框架补丁。作为研究智能体框架可靠性的早期工作，本文强调了将运行时轨迹与框架实现对齐以实现诊断感知修复的价值，为提升大语言模型智能体的可靠性提供了新的视角。

参考文献

- [1] J. Li, X. Xiao, Y. Zhang, C. Liu, L. Zhao, X. Liao, Y. Ji, J. Wang, J. Gu, Y. Ge, W. Xu, X. Fang, X. Xu, T. Zhao, Y. Kim, T. Wang, J. Hamm, S. Krishnaswamy, J. Huan, and C. Reddy, "Agent harness engineering: A survey," 2026. [Online]. Available: <https://openreview.net/pdf?id=eONq7FdiHa>
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *The Twelfth International Conference on Learning Representations*, 2024, oral presentation. [Online]. Available: <https://openreview.net/forum?id=VTF8yNQM66>
- [3] OpenAI, "SWE-bench Verified," <https://www.swebench.com/>, 2024.
- [4] M. A. Merrill *et al.*, "Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces," 2026.
- [5] G. Mialon *et al.*, "GAIA: A benchmark for general AI assistants," 2023.
- [6] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, "AppWorld: A controllable world of apps and people for benchmarking interactive coding agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 16 022–16 076.
- [7] D. Zhang, "AgentDevel: Reframing Self-Evolving LLM Agents as Release Engineering," 2026. [Online]. Available: <https://arxiv.org/abs/2601.04620>
- [8] Z. Hao, H. Wang, J. Luo, J. Zhang, Y. Zhou, Q. Lin, C. Wang, H. Dong, and J. Chen, "ReCreate: Reasoning and creating domain agents driven by experience," in *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*, 2026.
- [9] G. Fang, V. Isahagian, K. Jayaram, R. Kumar, V. Muthusamy, P. Oum, and G. Thomas, "Trajectory-informed memory generation for self-improving agent systems," *arXiv preprint arXiv:2603.10600*, 2026.
- [10] S. Zhang, M. Yin, J. Zhang, J. Liu, Z. Han, J. Zhang, B. Li, C. Wang, H. Wang, Y. Chen, and Q. Wu, "Which Agent causes task failures and when? on automated failure attribution of LLM Multi-Agent systems," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, pp. 1–12, spotlight paper.
- [11] I. Bouzenia and M. Pradel, "Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories," in *Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025, pp. 1–12.
- [12] N. Islam, R. S. Ayon, D. G. Thomas, S. Ahmed, and M. Wardat, "When agents fail: A comprehensive study of bugs in llm agents with automated labeling," *arXiv preprint arXiv:2601.15232*, 2026.
- [13] OpenAI, "A practical guide to building agents," <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>, 2025, accessed: 2026-05-09.
- [14] Anthropic, "Effective context engineering for AI agents," <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>, Sep. 2025, accessed: 2026-04-30.
- [15] C. Zhang, J. Yang, D. Yan, S. Yang, and Y. Chen, "Automated breakpoint generation for debugging," *J. Softw.*, vol. 8, no. 3, pp. 603–616, 2013.
- [16] Y. Chen, S. Zhang, Q. Guo, L. Li, R. Wu, and T. Chen, "Deterministic replay: A survey," *ACM Computing Surveys (CSUR)*, vol. 48, no. 2, pp. 1–47, 2015.
- [17] M. Monperrus, "Automatic software repair: A bibliography," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 17:1–17:24, 2018. [Online]. Available: <https://doi.org/10.1145/3105906>
- [18] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1482–1494. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00129>
- [19] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 2023, pp. 1430–1442. [Online]. Available: <https://doi.org/10.1109/ICSE48619.2023.00125>
- [20] Y.-A. Xiao, P. Gao, C. Peng, and Y. Xiong, "Improving the Efficiency of LLM Agent Systems through Trajectory Reduction," 2025. [Online]. Available: <https://arxiv.org/abs/2509.23586>
- [21] F. Lin, S. Chen, R. Fang, H. Wang, and T. Lin, "Stop Wasting Your Tokens: Towards Efficient Runtime Multi-Agent Systems," 2025. [Online]. Available: <https://arxiv.org/abs/2510.26585>
- [22] R. Nanda, C. Maddila, S. Jha, E. M. Khan, M. Paltenghi, and S. Chandra, "Wink: Recovering from misbehaviors in coding agents," *arXiv preprint arXiv:2602.17037*, 2026.
- [23] S. Liu, Y. Chen, R. Krishna, S. Sinha, J. Ganhotra, and R. Jabbardand, "Process-centric analysis of agentic software systems," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 1961–1988, 2026.
- [24] W. Wang, P. Kattakinda, and S. Feizi, "Maestro: Joint Graph & Config Optimization for Reliable AI Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2509.04642>
- [25] R. Costa, "Instruction-Level Weight Shaping: A Framework for Self-Improving AI Agents," 2025. [Online]. Available: <https://arxiv.org/abs/2509.00251>
- [26] Z. Pei, H.-L. Zhen, S. Kai, S. J. Pan, Y. Wang, M. Yuan, and B. Yu, "SCOPE: Prompt evolution for enhancing agent effectiveness," 2025.
- [27] J. Ni *et al.*, "Trace2Skill: Distill trajectory-local lessons into transferable agent skills," 2026.
- [28] Anthropic, "Effective harnesses for long-running agents," <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, 2025, accessed: 2026-05-09.
- [29] X. J. Wang, H. Bai, Y. Sun, H. Wang, S. Zhang, W. Hu, M. Schroder, B. Mutlu, D. Song, and R. D. Nowak, "The long-horizon task mirage? diagnosing where and why agentic systems break," 2026.
- [30] MiroMind Team, S. Bai, L. Bing, C. Chen, G. Chen, Y. Chen, Z. Chen, Z. Chen, J. Dai, X. Dong, W. Dou, Y. Deng, Y. Fu, J. Ge, C. Han, T. Huang, Z. Huang, J. Jiao, S. Jiang, T. Jiao, X. Jian, L. Lei, R. Li, G. Luo, T. Li, X. Lin, Z. Liu, Z. Li, J. Ni, Q. Ren, P. Sun, S. Su, C. Tao, B. Wang, W. Wang, H. Wang, J. Wang, J. Wang, J. Wang, L. Wang, S. Wang, W. Wang, Z. Wang, J. Xu, S. Xing, C. Yang, H. Ye, J. Yu, Y. Yu, M. Zhong, T. Zhao, X. Zhu, Y. Zhou, Y. Zhang, and Z. Zhu, "MiroThinker: Pushing the performance boundaries of open-source research agents via model, context, and interactive scaling," 2025, released as the MiroFlow agentic framework at <https://github.com/MiroMindAI/MiroFlow>.
- [31] Skywork AI, "DeepResearchAgent: A hierarchical multi-agent framework for deep research," <https://github.com/SkyworkAI/DeepResearchAgent>, 2025, accessed: 2026-05-30.
- [32] A. Roucher, C. Fourier, L. Tunstall, and L. von Werra, "Open-source DeepResearch: Freeing our search agents," <https://huggingface.co/blog/open-deep-research>, 2025, hugging Face blog; accessed 2026-05-30.
- [33] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.15793>
- [34] Harbor Framework Contributors, "Harbor: A containerized framework for agent benchmarking," <https://github.com/harbor-framework/harbor>, 2026, terminus-2 terminal-agent harness; accessed 2026-05-30.
- [35] T. Ma, Y. Chen, V. Anand, A. Cornacchia, A. R. Faustino, G. Liu, S. Zhang, H. Luo, S. A. Fahmy, Z. A. Qazi, and M. Canini, "MAESTRO: Multi-Agent Evaluation Suite for Testing, Reliability, and Observability," 2026. [Online]. Available: <https://arxiv.org/abs/2601.00481>

- [36] D. Guo, J. Wu, and S. M. Yiu, "Agenteval: Dag-structured step-level evaluation for agentic workflows with error propagation tracking," *arXiv preprint arXiv:2604.23581*, 2026.
- [37] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," in *The Fourteenth International Conference on Learning Representations*, 2026, oral presentation. [Online]. Available: <https://openreview.net/forum?id=RQm2KQTM5r>
- [38] X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang, "Gödel agent: A self-referential agent framework for recursively self-improvement," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 27 890–27 913. [Online]. Available: <https://aclanthology.org/2025.acl-long.1354/>
- [39] M. Robeyns, M. Szummer, and L. Aitchison, "SICA: A self-improving coding agent," in *ICLR 2025 Workshop on Scaling Self-Improving Foundation Models*, 2025, oral presentation. [Online]. Available: <https://openreview.net/forum?id=rShJCyLsOr>
- [40] J. Zhang, S. Hu, C. Lu, R. T. Lange, and J. Clune, "Darwin gödel machine: Open-ended evolution of self-improving agents," in *The Fourteenth International Conference on Learning Representations*, 2026, poster presentation. [Online]. Available: <https://openreview.net/forum?id=pUpzQZTvGY>
- [41] W. Wang, P. Piękos, N. Li, F. Laakom, Y. Chen, M. Ostaszewski, M. Zhuge, and J. Schmidhuber, "Huxley-gödel machine: Human-level coding agent development by an approximation of the optimal self-improving machine," in *The Fourteenth International Conference on Learning Representations*, 2026, oral presentation. [Online]. Available: <https://openreview.net/forum?id=TOEiEuhOOL>
- [42] SWE-agent Contributors, "mini-swe-agent: A minimal LLM agent for software engineering," <https://github.com/SWE-agent/mini-swe-agent>, 2024, accessed: 2026-05-30.
- [43] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, H. H. Tran, F. Li, R. Ma, M. Zheng, B. Qian, Y. Shao, N. Muennighoff, Y. Zhang, B. Hui, J. Lin, R. Brennan, H. Peng, H. Ji, and G. Neubig, "OpenHands: An open platform for AI software developers as generalist agents," in *The Thirteenth International Conference on Learning Representations*, 2025, poster presentation. [Online]. Available: <https://openreview.net/forum?id=OJd3ayDDoF>
- [44] Trae Research Team, P. Gao, Z. Tian, X. Meng, X. Wang, R. Hu, Y. Xiao, Y. Liu, Z. Zhang, J. Chen, C. Gao, Y. Lin, Y. Xiong, C. Peng, and X. Liu, "Trae Agent: An LLM-based agent for software engineering with test-time scaling," 2025. [Online]. Available: <https://arxiv.org/abs/2507.23370>
- [45] CUGA Project Contributors, "CUGA: A computer-using generalist agent," <https://github.com/cuga-project/cuga-agent>, 2025, accessed: 2026-05-30.
- [46] OpenCode Contributors, "OpenCode: An open-source coding agent," <https://github.com/anomalyc0/opencode>, 2024, accessed: 2026-05-30.
- [47] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn, "Meta-harness: End-to-end optimization of model harnesses," 2026.
- [48] Y. Wang, W. Wu, J. Wang, and Q. Wang, "From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems," 2026. [Online]. Available: <https://arxiv.org/abs/2602.23701>
- [49] M. Ma, J. Zhang, F. Yang, Y. Kang, Q. Lin, T. Yang, S. Rajmohan, and D. Zhang, "DoVer: Intervention-Driven Auto Debugging for LLM Multi-Agent Systems," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2026, pp. 1–16.
- [50] R. Lopopolo, "Harness engineering: Leveraging Codex in an agent-first world," <https://openai.com/index/harness-engineering/>, Feb. 2026, accessed: 2026-04-30.
- [51] L. Pan, L. Zou, S. Guo, J. Ni, and H.-T. Zheng, "Natural-language agent harnesses," 2026.
- [52] H. Liu, C. Shou, X. Liu, H. Wen, Y. Chen, R. J. Fang, and Y. Feng, "Synthesizing multi-agent harnesses for vulnerability discovery," 2026.
- [53] X. Lou, M. Lázaro-Gredilla, A. Dedieu, C. Wendelken, W. Lehrach, and K. P. Murphy, "AutoHarness: Improving LLM agents by automatically synthesizing a code harness," 2026.
- [54] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," 2026.
- [55] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *The Thirteenth International Conference on Learning Representations*, 2025.
- [56] J. Zhang *et al.*, "AFlow: Automating agentic workflow generation," in *The Thirteenth International Conference on Learning Representations*, 2025, oral presentation. [Online]. Available: <https://openreview.net/forum?id=z5uVAKwmjf>
- [57] Z. Li *et al.*, "AutoFlow: Automated workflow generation for large language model agents," 2024.
- [58] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li, "AgentSquare: Automatic LLM agent search in modular design space," 2024.
- [59] G. Zhang, L. Niu, J. Fang, K. Wang, L. Bai, and X. Wang, "MaAS: Multi-agent architecture search via agentic supernet," in *Proceedings of the 42nd International Conference on Machine Learning*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.04180>
- [60] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," in *The Twelfth International Conference on Learning Representations*, 2024, poster presentation. [Online]. Available: <https://openreview.net/forum?id=qV83K9d5WB>
- [61] C. Qian, C. Han, Y. R. Fung, Y. Qin, Z. Liu, and H. Ji, "CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.462/>
- [62] R. Wang, X. Han, L. Ji, S. Wang, T. Baldwin, and H. Li, "ToolGen: Unified tool retrieval and calling via generation," in *The Thirteenth International Conference on Learning Representations*, 2025.
- [63] X. Liu, X. Luo, L. Li, G. Huang, J. Liu, and H. Qiao, "SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support," in *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2026, industry Track. [Online]. Available: <https://arxiv.org/abs/2604.08618>
- [64] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig, "Agent workflow memory," 2024.
- [65] X. Tang *et al.*, "Agent KB: Leveraging cross-domain experience for agentic problem solving," 2025.
- [66] W. Xu *et al.*, "A-MEM: Agentic memory for LLM agents," 2025.
- [67] R. Salama *et al.*, "MemInsight: Autonomous memory augmentation for LLM agents," 2025.
- [68] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, "Agentic context engineering: Evolving contexts for self-improving language models," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=eC4ygDs02R>
- [69] Z. Weng, A. Antoniadis, D. Nathani, Z. Zhang, X. Pu, and X. E. Wang, "Group-evolving agents: Open-ended self-improvement via experience sharing," 2026.
- [70] Y. Hu, Y. Cai, Y. Du, X. Zhu, X. Liu, Z. Yu, Y. Hou, S. Tang, and S. Chen, "Self-evolving multi-agent collaboration networks for software development," 2024.
- [71] SkyworkAI, "Autogenesis: A self-evolving agent protocol," 2026.
- [72] Y. He, J. Liu, Y. Liu, Y. Li, T. Cao, Z. Hu, X. Xu, and B. Hooi, "EvoTest: Evolutionary test-time learning for self-improving agentic systems," in *The Fourteenth International Conference on Learning Representations*, 2026. [Online]. Available: <https://arxiv.org/abs/2510.13220>
- [73] Y. Wan, T. Fang, Z. Li, Y. Huo, W. Wang, H. Mi, D. Yu, and M. R. Lyu, "Inference-time scaling of verification: Self-evolving deep research agents via test-time rubric-guided verification," 2026.
- [74] J. Wang, Y. Wang, M. Chen, X. Xie, C. Chen, F. Mu, Z. Liu, and Q. Wang, "A survey for llm agent trajectory analysis: From failure attribution to enhancement," 2026.
- [75] H. Han *et al.*, "SWE-TRACE: Optimizing long-horizon SWE agents through rubric process reward models and heuristic test-time scaling," 2026.
- [76] L. Zhang and M. Agrawala, "View-oriented conversation compiler for agent trace analysis," 2026.
- [77] T. Mehtiyev and W. Assunção, "Beyond resolution rates: Behavioral drivers of coding agent success and failure," 2026.
- [78] Y. Ge, L. Xie, Z. Li, Y. Pei, and T. Zhang, "Who is Introducing the Failure? Automatically Attributing Failures of Multi-Agent Systems via Spectrum Analysis," in *Proceedings of the 31st ACM Joint European*

Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2026, pp. 1–12.

- [79] G. Zhang, J. Wang, J. Chen, W. Zhou, K. Wang, and S. Yan, “AgentTracer: Who is inducing failure in the LLM agentic systems?” 2025.
- [80] H. Mulian, S. Zeltyn, I. Levy, L. Galanti, A. Yaeli, and S. Shlomov, “AgentFixer: From failure detection to fix recommendations in agentic systems,” in *Proceedings of the 2026 International Workshop on Agentic Engineering*, ser. AGENT ’26. New York, NY, USA: Association for Computing Machinery, 2026, pp. 96–103. [Online]. Available: <https://doi.org/10.1145/3786167.3788427>